





HAI716I – Programmation Orientée Agents

Rapport Projet n°1 : Robot Réconfort

Adnan FAIZE Eya SELMI

Résumé : Ce rapport présente notre robot de réconfort : un agent qui ne connaît pas les murs de l'appartement, les découvre en se déplaçant, consulte un dictionnaire pour comprendre l'émotion d'un résident, fouille une armoire et lui apporte un objet. Nous l'avons programmé en Python, sans bibliothèque externe. Toutes les mesures citées ont été faites sur les cinq cartes fournies.

Année 1 Master Informatique parcours IMAGINE
2026 – 2027

1 Architecture

Le programme s'exécute avec la commande suivante.

```
console                                                    Bash
1  python main.py <chemin_carte> <chemin_scenario> <dossier_donnees> <chemin_trace>
```

1.1 Les fichiers et leur rôle

`main.py` charge et vérifie les quatre fichiers avec `reconfort_io.py`, crée le simulateur (dans `monde.py`) et l'agent (dans `robot.py`), puis donne au robot les demandes une par une. Figure 1

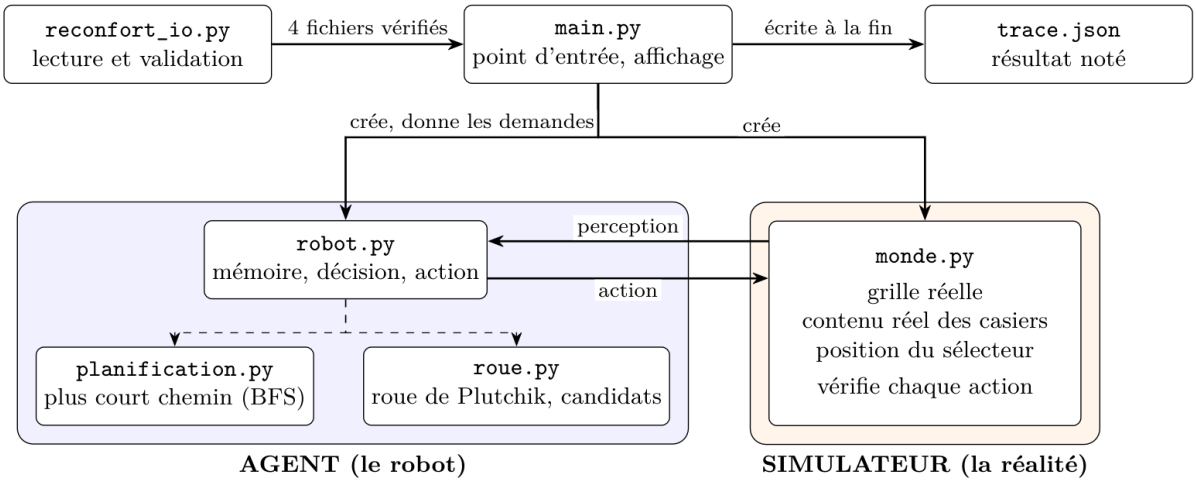


Figure 1: Architecture des fichiers et interactions entre les composants du système

Pour chaque demande, le robot appelle `traiter_demande` qui enchaîne six phases :
1. aller au dictionnaire;
2. `"CONSULTER"` ;
3. aller à l'armoire;
4. fouiller avec `"CHERCHER"` ;
5. `"PRENDRE"` ;
6. aller au résident et `"DONNER"`

Chaque action est notée dans la trace.

1.2 Perception, mémoire, décision, action

	Où, dans le code	Ce que ça fait
Perception	`"Monde.percevoir()"` `"Monde.contenu_casier()"`	le monde renvoie le type des 4 cases voisines, et l'objet du casier courant si le robot est devant l'armoire
Mémoire	`"Robot.carte_mentale"` `"memoriser()"` `"memoire_armoire"` `"position_selecteur"`	le robot range ce qu'il a perçu : murs et cases libres, contenu des casiers déjà vus. Cette mémoire est gardée d'une demande à l'autre

Décision	`"analyser_message()"` `"prochaine_direction()"` `"candidats_repli()"`	lire l'émotion (règle 6.1) ; choisir le prochain pas par BFS ; ordonner les 15 casiers à fouiller (règle 6.2)
Action	`"aller_vers()"` `"traiter_demande()"` `"Monde.avancer/chercher/..."`	le robot demande l'action au monde, qui la vérifie et l'exécute ; le pas est écrit dans la trace

1.3 Pourquoi ce découpage

- **Le simulateur et l'agent sont dans deux fichiers différents.** Seul `monde.py` lit le champ `grille` de la carte et le champ `objet` des casiers. Le robot ne connaît que les dimensions et les positions de l'armoire, du dictionnaire et des résidents. Il apprend tout le reste par `percevoir()` et `contenu_casier()`, comme l'exige l'énoncé (sections 3.2 et 3.3).
- **Le simulateur refuse les actions impossibles.** `avancer` dans un mur, `chercher` au-delà de la ligne 0 ou 2, `donner` loin du résident : chaque cas lève l'exception `ActionIllegale`. Une erreur dans l'agent arrête donc le programme au lieu de produire une trace fausse.
- **Le BFS ne connaît pas la carte.** `plus_court_chemin` reçoit une fonction `est_traversable(case)`. Le robot lui donne sa carte mentale ; nos tests lui donnent de petites grilles dessinées à la main. Le même code sert aux deux, et pourra resservir au projet 2.
- **La roue est isolée dans `roue.py`.** La distance sur la roue, le tri des candidats et le trajet du sélecteur ne dépendent ni de la grille ni du robot, ce qui permet de les tester seuls (tests de bouclage et de repli).

1.3.1 Limites de notre découpage

Nous les avons remarquées en relisant le code. (1) Le robot reçoit la carte complète, et c'est par discipline qu'il ne lit pas la grille : rien ne l'en empêche techniquement. Il aurait été plus sûr de lui donner une copie sans le champ `grille`. (2) Le robot écrit lui-même dans la trace : l'agent dépend donc du format de sortie. (3) `Monde.prendre()` ne vérifie pas que le casier courant contient bien l'objet demandé. Pendant l'intégration, un bug de notre sélecteur (qui ne montait jamais vers la ligne 0) a ainsi produit des `PRENDRE` faux sans que le simulateur ne réagisse. Nous ne l'avons découvert qu'en rejouant les traces contre la réalité.

2 Stratégie de replanification

Le principe. Au départ, le robot ne connaît aucun mur. Nous faisons l'**hypothèse optimiste** de l'annexe B.5 : une case jamais vue est supposée libre. La boucle de `aller_vers` est alors :

1. **percevoir** les 4 cases voisines ;
2. les **mémoriser** dans la carte mentale ;
3. si le robot est déjà à côté de la cible, s'arrêter ; sinon, **recalculer** un plus court chemin par BFS sur la carte mentale, jusqu'à une case voisine de la cible ;
4. faire **un seul pas** de ce chemin, puis recommencer.

Nous replanifions tout, à chaque pas, sans essayer de réparer l'ancien plan. C'est simple : il n'y a pas besoin de savoir quand un plan devient faux. Quand un mur est découvert, il est dans la carte mentale, et le BFS suivant passe ailleurs. Sur l'appartement 01, par exemple, le robot part vers l'est sur la ligne 6, découvre le mur de la colonne 7 en arrivant en `[6,6]`, et le BFS suivant le fait remonter par la ligne 2 (le comportement décrit à l'annexe D).

Le cas inaccessible. Comme la carte mentale suppose libre tout ce qui est inconnu, elle a toujours au moins autant de passages que la réalité. Si le BFS ne trouve aucun chemin sur cette carte optimiste, il n'en existe donc aucun dans la réalité (annexe B.6) : le robot peut conclure « résident inaccessible » sans visiter tout l'appartement.

Le coût, mesuré sur les cinq cartes :

Appartement	**01**	**02**	**03**	**04**	**05**
Taille	9× 13	11× 15	11× 17	13× 19	13× 21
Appels au BFS (un par pas)	134	199	231	203	215
Temps de calcul total	14 ms	18 ms	22 ms	20 ms	36 ms
Déplacements réels (`AVANCER`)	134	199	231	203	215
Déplacements si les murs étaient connus	120	179	207	169	201
Surcoût dû à l'exploration	+12 %	+11 %	+12 %	+20 %	+7 %

Un BFS visite au plus toutes les cases, soit $O(H \times W)$: 273 cases sur la plus grande carte. Même en le relançant à chaque pas, le calcul total reste sous 40 ms par scénario : **le coût en temps est négligeable**. Le vrai coût est en **nombre de pas** : l'exploration ajoute 7 à 20 % de déplacements par rapport à un robot qui connaîtrait le plan. Réparer le plan au lieu de tout recalculer ne changerait pas ce chiffre, car le trajet serait le même ; on économiserait seulement des calculs. Sur une carte beaucoup plus grande, nous ne relancerions le BFS que lorsqu'un mur découvert coupe le chemin prévu.

3 Critique de la règle de repli

La règle : Utilisation de la roue de Plutchik pour trouver des substituts (distance ≤ 2, triés par distance, écart d'intensité, intensité faible et sens horaire).

Critique : Cette approche est peu pertinente car :
- La distance sur la roue mesure la ressemblance émotionnelle et non le réconfort (ex. proposer des gants de ménage pour de la tristesse).
- Le sens horaire et le choix de privilégier les faibles intensités sont arbitraires.

Notre proposition : Remplacer la roue par une **table de repli manuelle** (ex. tristesse → confiance, joie), en testant d'abord d'autres intensités de la même émotion.

Changements nécessaires :
- `armoire_standard.json` : ajouter le champ `"replis"` .
- `reconfort_io.py` : valider la table et adapter les étiquettes de la trace (`table_1` , etc.).
- `roue.py` : utiliser cette table au lieu de calculer les distances sur la roue.

4 Trois échecs de notre robot

Échec 1 — Résident inaccessible : Le robot récupère l'objet avant de vérifier si le résident est joignable. S'il est bloqué, il garde l'objet en main et fait planter le programme à la demande suivante.
Piste : Vérifier l'accessibilité du résident avant de chercher l'objet, ou passer la demande en échec au lieu de stopper tout le scénario.

Échec 2 — Impasses par optimisme : Le robot s'engage dans des impasses (+160 % de pas) car il suppose tout libre et son BFS ignore les cases déjà visitées en cas d'égalité.
Piste : Privilégier les chemins traversant des zones déjà connues à distance égale.

Échec 3 — Fouille inefficace de l'armoire : Le robot visite les casiers candidats un par un sans optimiser son trajet global, générant des mouvements superflus.
Piste : Calculer au préalable le parcours minimal passant par tous les casiers à inspecter.